FLIGHT MANAGEMENT SYSTEM

Database Design & Implementation

Artyom Mkrtchyan | Mane Mazmandyan | Davit Arakelyan

French University in Armenia | April 2026

Team Contributions

Project Area	Member(s)
Project Planning & Requirements	Worked Together
Conceptual Design (ER Diagram)	Mane Mazmandyan
Logical Design & Normalization	Davit Arakelyan
Physical DB Implementation	Mane Mazmandyan
Views, Indexes, Triggers, SPs	Artyom Mkrtchyan
Frontend/UI Development	Artyom Mkrtchyan
Testing & Query Optimization	Davit Arakelyan
Final Report & Presentation	Worked Together

Challenges of Flight Managing Systems

Problem Description

- ✈️ Airlines manage a **complex network of data** (aircraft, flights, routes, passengers)
- 🔗 All data must stay **correctly connected** — everything depends on everything
- 🛬 Aircraft must **belong to the right airline**
- 🌍 Routes must **link valid airports only**
- ❓ Flights must **match both a route and an aircraft**
- ⚠️ Main challenge: keeping **data consistent and error-free**

Project Goal

- 🔄 Supports CRUD operations
- 📊 Allows analytical queries
- ⚙️ Includes automation
- 💾 Uses a relational database to manage airline data
- 🚀 Reliable and scalable solution

Introduction: Flight Management System

✈ Flights

Passengers

Global Connections

All managed by **one system !!!**

Conceptual Design

Main Data Objects

- Airlines – airline companies
- Aircraft – airplanes per airline
- Airports – departure/arrival locations
- Routes – airport connections
- Flights – scheduled operations
- Passengers
- Tickets

Business Rules

- Each aircraft belongs to one airline
- Each flight links to a valid route + aircraft
- Each route connects two existing airports
- Each ticket ties to a passenger + flight

10
Core Tables

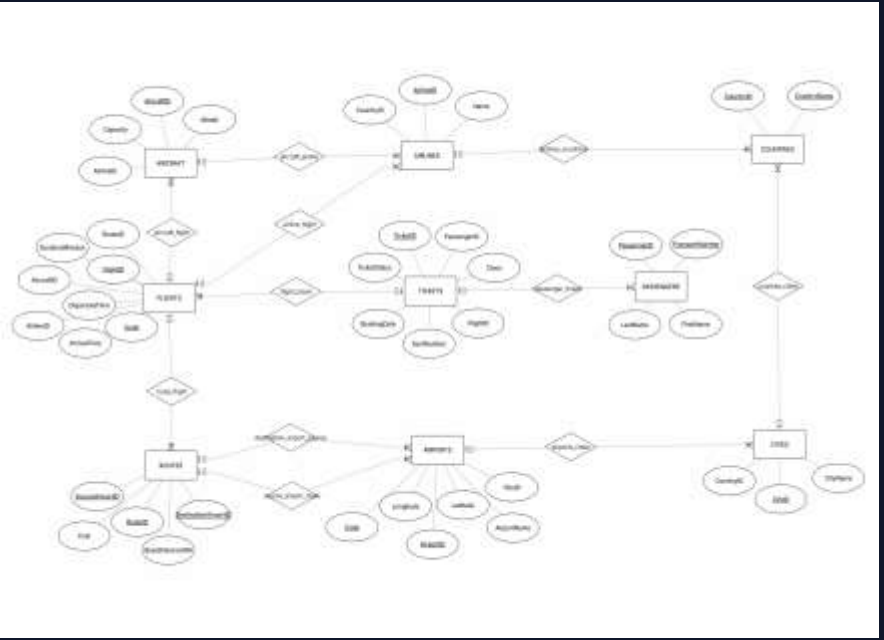
9
Entities

11
Relationships

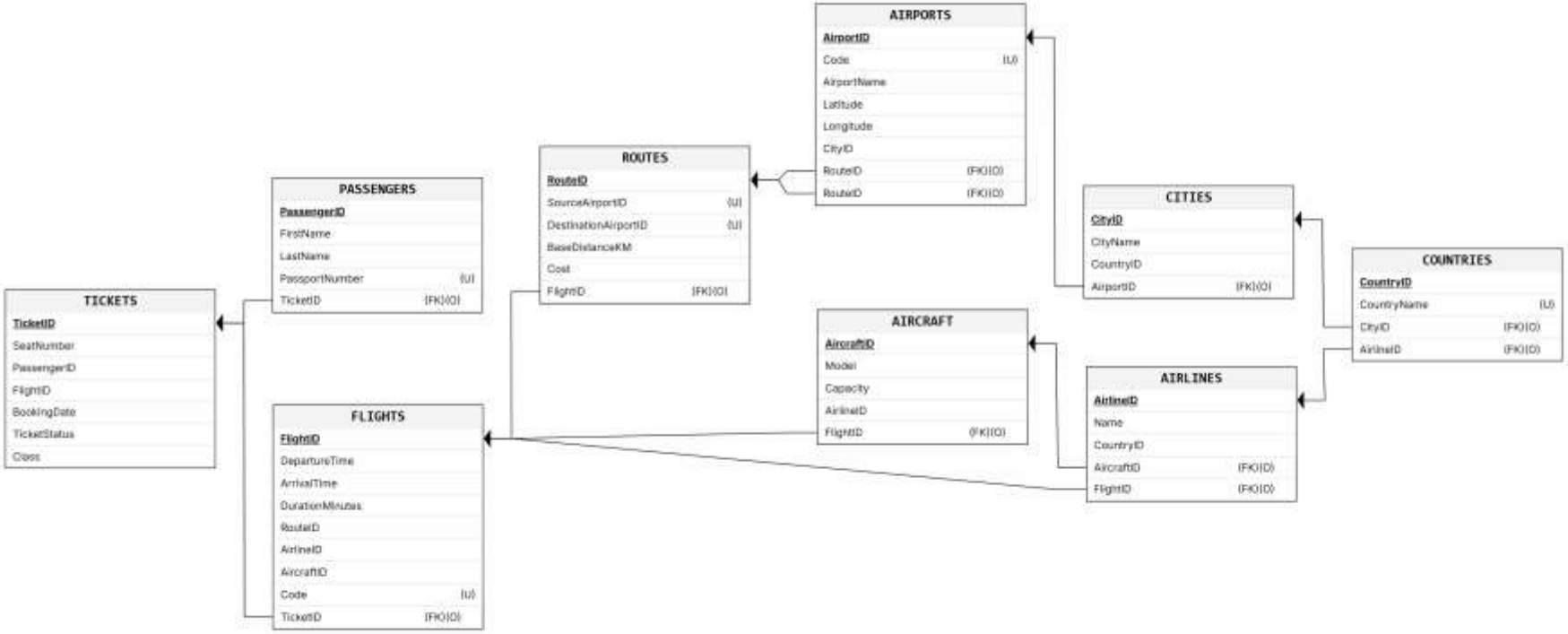
ER Diagram & Relational Schema

ER Diagram

- Airlines (AirlineID PK, Name, CountryID FK)
- Aircraft (AircraftID PK, Model, Capacity, AirlineID FK)
- Countries (CountryID PK, CountryName)
- Cities (CityID PK, CityName, CountryID FK)
- Airports (AirportID PK, Code, AirportName, CityID FK, Lat, Lng)
- Routes (RouteID PK, SourceAirportID FK, DestAirportID FK, DistKM, Cost)
- Flights (FlightID PK, RouteID FK, AirlineID FK, AircraftID FK, Times, Code)
- Passengers (PassengerID PK, FirstName, LastName, PassportNumber)
- Tickets (TicketID PK, PassengerID FK, FlightID FK, Seat, Date, Status, Class)



Relational Schema



Relationships & Constraints

Key Relationships (all 1:N)

- Airline → many Aircraft
- Airline → many Flights
- Country → Cities → Airports
- Route → many Flights
- Passenger → many Tickets
- Flight → many Tickets

Integrity Constraints

- PKs uniquely identify each record
- FKs enforce referential integrity
- PassportNumber is unique per passenger
- Every ticket references a valid flight

Normalization Process

1NF

- No repeating groups
- Each field = one value
- Primary key defined

Passenger → Name, Passport

2NF

- Already in 1NF
- All non-key attributes fully depend on the primary key
- No partial dependency

Flight → all data depends on FlightID

3NF

- Already in 2NF
- No transitive dependencies
- Separate related data into tables

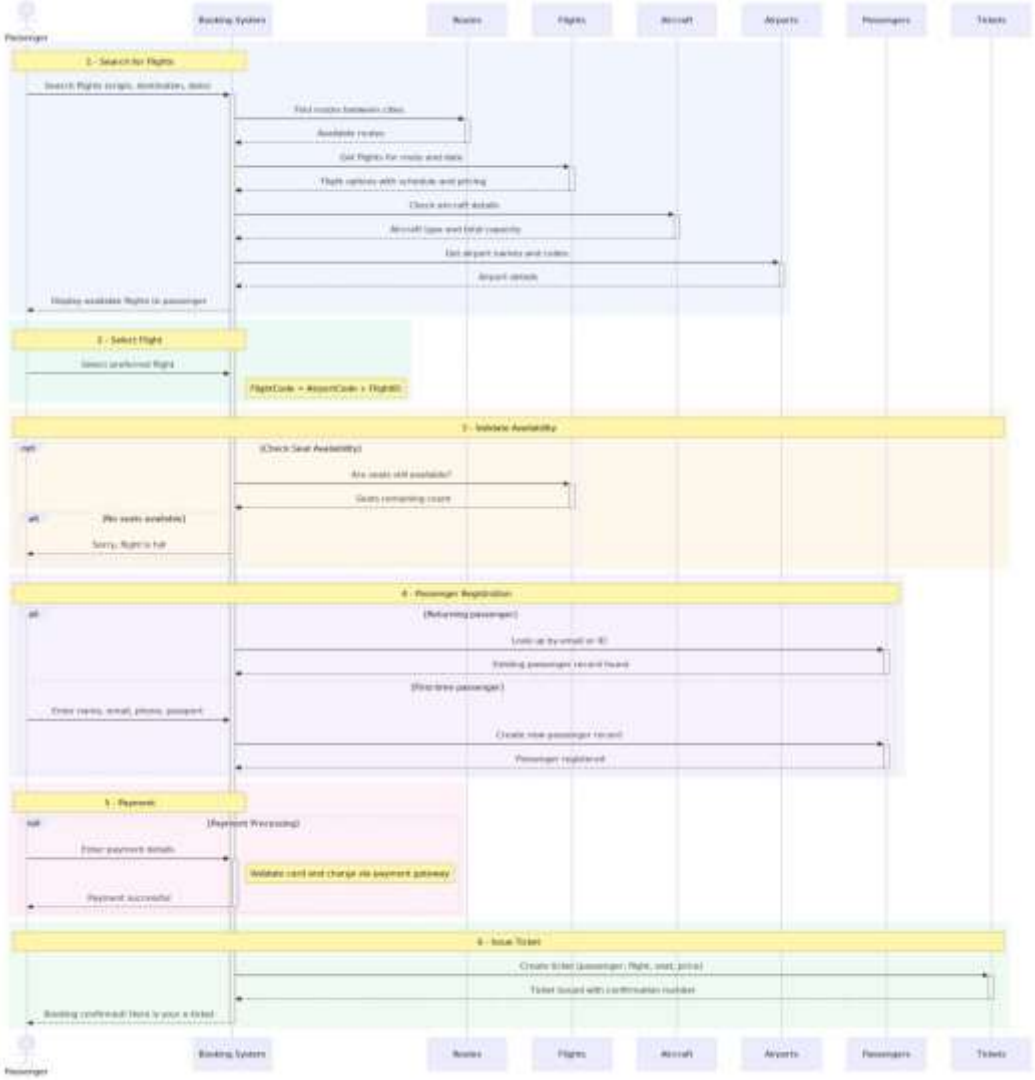
Country stored separately from City

Sequence Diagram

Shows messages between system components

- Example: Booking a Ticket

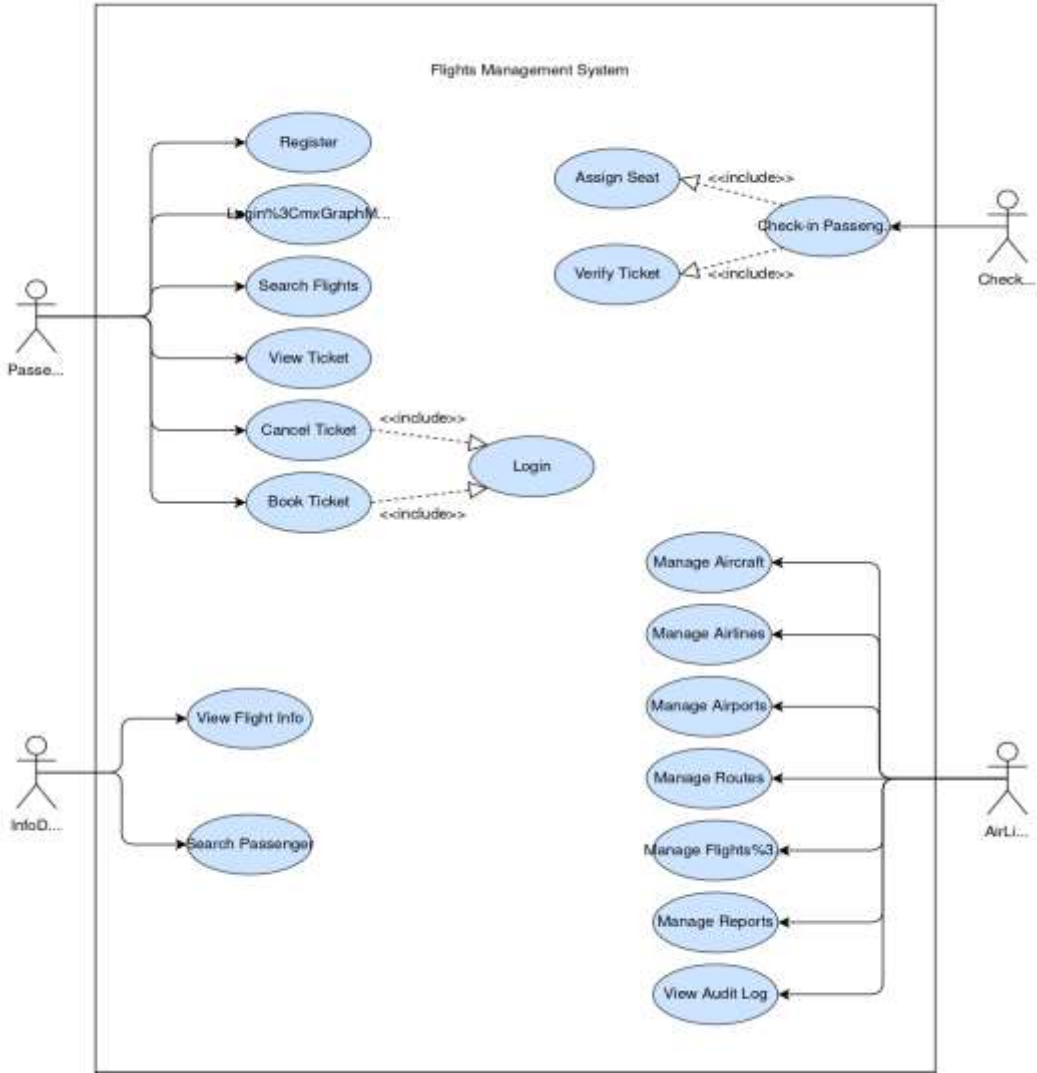
1. User requests flight
2. System queries database
3. Database returns available flights
4. User selects flight → System issues ticket



Use Case Diagram

Shows how different users interact with the system.

1. Passenger – Search, Book, Cancel Tickets
2. Airline Staff – Manage Flights & Routes …
3. CheckIn – Assign Seat and Verify Seat
4. InfoDesk – Search Passenger, View Flight Info

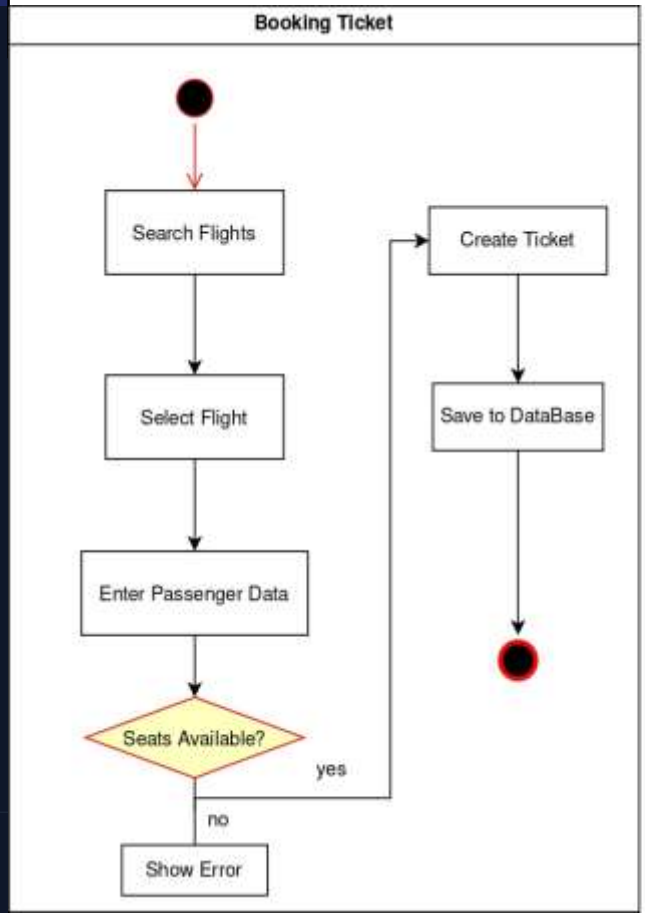


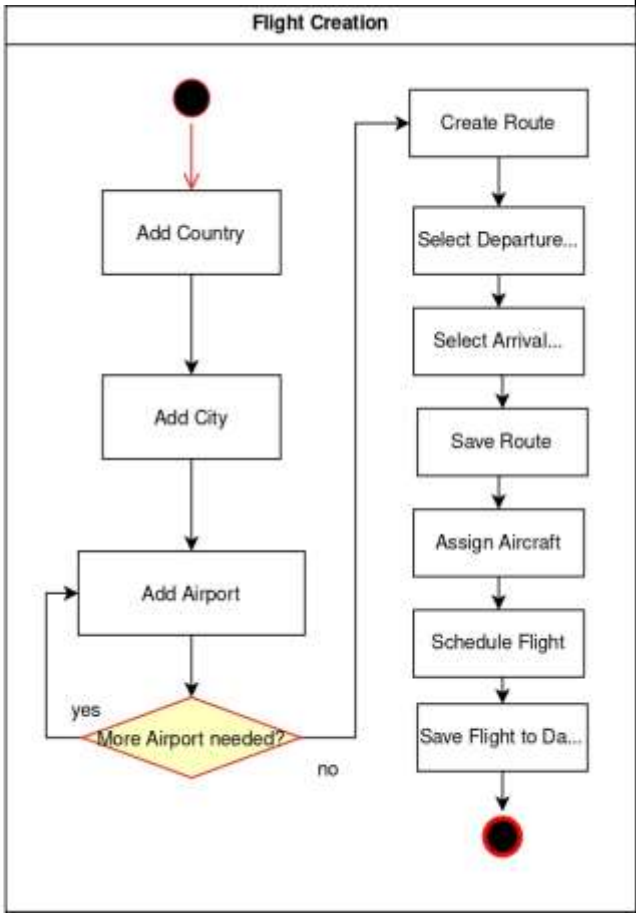
Activity Diagram

Shows step-by-step process

- Highlights decisions and parallel tasks
- Example: Ticket Booking

1. User selects flight
2. System checks availability
3. User confirms
4. Ticket is issued





Relational Algebra Queries

Union

Combines airline names with city names where they operate flights

```
SELECT Name AS EntityName
FROM Airlines
UNION
SELECT ci.CityName
FROM Flights f
JOIN Routes r ON ...
JOIN Cities ci ON ...
```

Intersection

Passengers with tickets for flights > 180 min AND departing from JFK

```
SELECT PassengerID
FROM Tickets t
JOIN Flights f ON ...
WHERE DurationMinutes > 180
INTERSECT
SELECT t.PassengerID
FROM Tickets t ...
WHERE a.Code = 'JFK'
```

Difference

Finds all airlines without any aircraft by selecting AirlineIDs from the Airlines table that do not appear in the Aircraft table.

Relational Algebra Queries

Selection

Retrieves all flights from the Flights table with a departure time later than March 25, 2026, at 12:00.

Projection

Selects only the FlightCode and DepartureTime columns from the Flights table, showing limited information for all flights

Join

Joins the Flights table with Routes and Airports (twice) to display each flight along with its route details and the names of its source and destination airports.

SQL Scripts Overview

DDL

CREATE TABLE with PKs,
FKs, and constraints

DML

INSERT data for cities,
airports, airlines, etc.

DQL

SELECT with JOINs for
flight and airline queries

DCL

GRANT/DENY roles:
InfoDesk, Staff,
Airline Manager

Security Roles Defined

InfoDesk_Agent (read-only) | Airport_Staff (select + limited update) | Airline_Internal_Staff (full CRUD)

Views, Indexes, Triggers & SPs

Views

- VFlightBoard – flight departure board
- VSM_RoutePrDRSt – route ticket sales
- PassengerManagement – manifest view

Indexes

- IX_Flights_RouteID, AircraftID, AirlineID
- IX_Tickets_FlightID
- IX_Tickets_Status_Date (composite)

Triggers

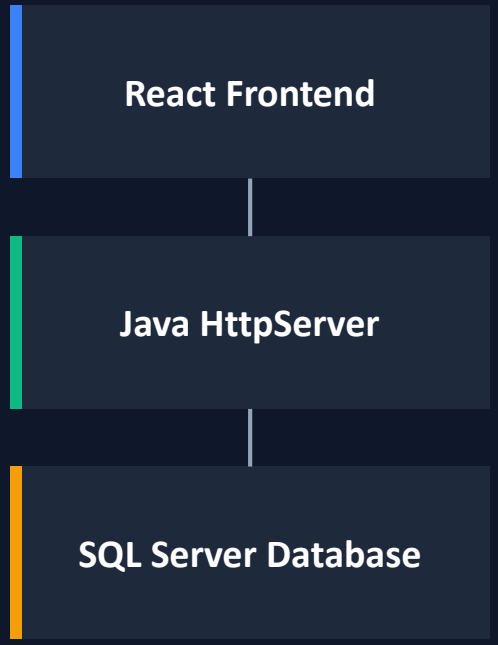
- tg_AfterTicketUpdate – audit log
- trg_CheckFlightCapacity – prevents overbooking

Stored Procedures

- sp_FindFlightsByCities – search flights
- Automatic ticket booking

Database Deployment

- Built in SQL Server Management Studio
- TCP/IP enabled with firewall for remote access
- Frontend: JavaScript/React with interactive UI
- Backend: Java HttpServer handling HTTP requests
- SQL queries return JSON data to the frontend
- Users can view, add, and delete records



Database Management

Back to the map

Aircraft | Airlines | Airports | Cities | Countries | Flights | Passengers | Routes | TicketAuditLog | Tickets

Search...

Delete Selected (0)

+ Add row

AIRLINEID	DEPARTURETIME	AIRCRAFTID	DURATIONMINUTES	FLIGHTCODE	FLIGHTID	ROUTEID	ARRIVALTIME	ACTIONS
162	2026-03-20 08:30:00.0	1106	135	UZJ435	5	394	2026-03-20 10:45:00.0	
168	2026-03-20 14:15:00.0	1107	165	OOG401	6	395	2026-03-20 17:00:00.0	
162	2026-03-20 20:00:00.0	1108	130	XPZ373	7	396	2026-03-20 22:10:00.0	
163	2026-03-21 06:45:00.0	1109	150	QNH123	8	397	2026-03-21 09:15:00.0	
169	2026-03-21 11:30:00.0	1110	190	YEQ282	9	398	2026-03-21 14:40:00.0	
163	2026-03-21 16:00:00.0	1111	110	RLH346	10	399	2026-03-21 17:50:00.0	
164	2026-03-21 21:10:00.0	1112	190	FCQ578	11	400	2026-03-22 00:20:00.0	
164	2026-03-22 07:40:00.0	1113	160	THG011	12	401	2026-03-22 10:20:00.0	
187	2026-03-22 13:20:00.0	1114	270	NPR640	13	402	2026-03-22 17:50:00.0	

Fastest
Cheapest
BFS for K
Critical
MST
Budget
DB
CANADA
UNITED STATES
MEXICO
GUATEMALA
VENEZUELA
COLOMBIA
GUYANA
PERU
BRAZIL
BOLIVIA
PARAGUAY
URUGUAY
ARGENTINA
NORWAY
FINLAND
RUSSIA
BELARUS
POLAND
UKRAINE
KAZAKHSTAN
MONGOLIA
ROMANIA
BULGARIA
TURKEY
UZBEKISTAN
TURKMENISTAN
SPAIN
TUNISIA
MOROCCO
ALGERIA
LIBYA
EGYPT
IRAN
AFGHANISTAN
PAKISTAN
NEPAL
INDIA
CHINA
MAURITANIA
MALI
NIGER
CHAD
SUDAN
ERITREA
YEMEN
NIGERIA
SOUTH SUDAN
ETHIOPIA
KENYA
RWANDA
TANZANIA
ANGOLA
ZAMBIA
MOZAMBIQUE
NAMIBIA
SOUTH AFRICA
AIRPORTS 51
ROUTES 193
STATUS LIVE

Conclusions & Results

- Successfully designed and implemented a relational database for airline operations
- Core tables structured with PK/FK constraints ensuring referential integrity
- Views, Triggers, and Stored Procedures enable efficient query execution
- Automated processes track ticketing history through audit logs
- System provides a robust, consistent, and extensible solution

github.com/Artyomkrtchyan/FlightManagmentSystem

Thank You!